

■ COMPLETE NOTES - TOPIC 15

CSS Flex Box

One-dimensional layouts, alignment, responsive components aur advanced
sizing - Basic se Advanced tak

Web Development Course - Shauray

display flex

main axis

cross axis

flex-direction

justify-content

align-items

flex-wrap

gap

flex-grow

flex-shrink

flex-basis

order

Flexbox Kya Hai?

One-dimensional layout system for rows or columns

SIMPLE DEFINITION

Flexbox CSS ka layout model hai jo items ko ek direction mein arrange, align, distribute aur resize karne ke liye use hota hai. Ek time par Flexbox mainly one dimension handle karta hai: row ya column.

Flexbox se navbars, button groups, cards row, centered content, sidebar layouts, equal height items, responsive components aur vertical/horizontal alignment easily handle hota hai. Pehle jo layout hacks float, inline-block aur table display se bante the, Flexbox unhe clean aur predictable bana deta hai.

■ Flex Container and Flex Items

Term	Meaning	Example
Flex Container	Jis parent par display: flex lagta hai	.navbar, .cards, .toolbar
Flex Items	Container ke direct children	links, cards, buttons
Main Axis	Primary direction jisme items arrange hote hain	row mein horizontal
Cross Axis	Main axis ke perpendicular direction	row mein vertical

● ● ● first flex layout

```
.container {  
  display: flex;  
  gap: 1rem;  
}
```

BASIC FLEX DEMO



- **Core Idea:** Parent par `display: flex` lagate hi uske direct children flex items ban jaate hain. Grandchildren automatically flex items nahi bante.

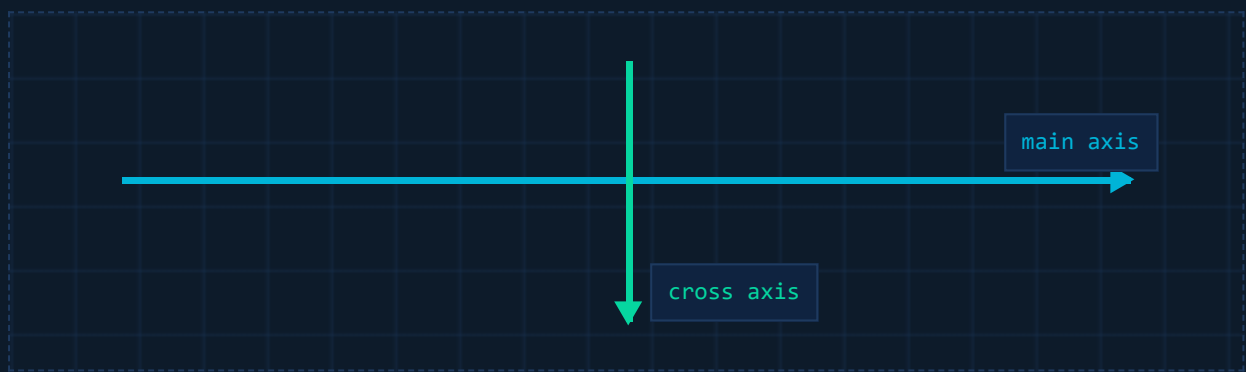
Main Axis and Cross Axis

Flexbox samajhne ka sabse important concept

AXIS CONCEPT

Flexbox mein alignment properties axis ke according kaam karti hain. `justify-content` main axis par items distribute karta hai. `align-items` cross axis par items align karta hai.

AXIS GRAPH - FLEX-DIRECTION: ROW



■ Axis Depends on flex-direction

flex-direction	Main Axis	Cross Axis
row	Left to right	Top to bottom
row-reverse	Right to left	Top to bottom
column	Top to bottom	Left to right
column-reverse	Bottom to top	Left to right

■ **Common Confusion:** `justify-content` hamesha horizontal nahi hota. Agar `flex-direction: column` hai, to `justify-content` vertical direction mein kaam karega.

flex-direction

Items row mein jayenge ya column mein

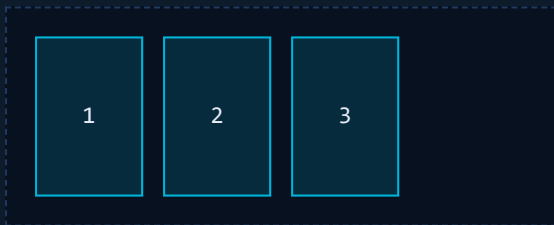
DEFINITION

flex-direction main axis ki direction decide karta hai. Default value **row** hoti hai, isliye flex items normally left to right arrange hote hain.

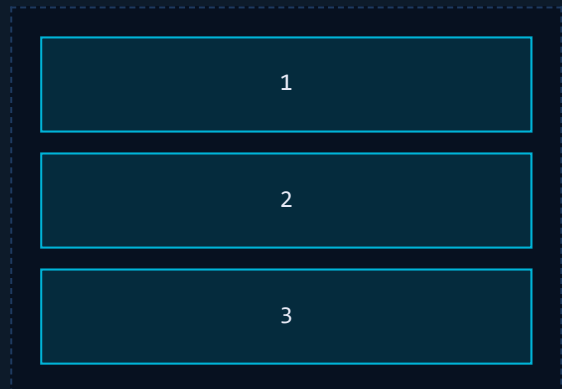
● ● ● direction values

```
.row { flex-direction: row; }  
.row-reverse { flex-direction: row-reverse; }  
.column { flex-direction: column; }  
.column-reverse { flex-direction: column-reverse; }
```

ROW



COLUMN



■ **Practical Use:** Mobile par component ko column aur desktop par row banana Flexbox ka common responsive pattern hai.

justify-content

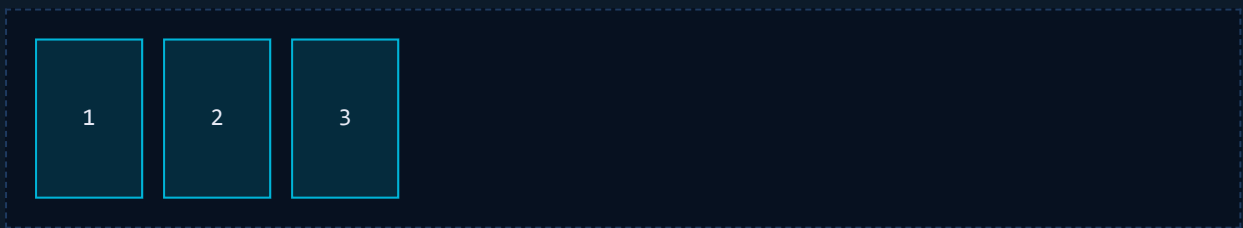
Main axis par items ko distribute karna

DEFINITION

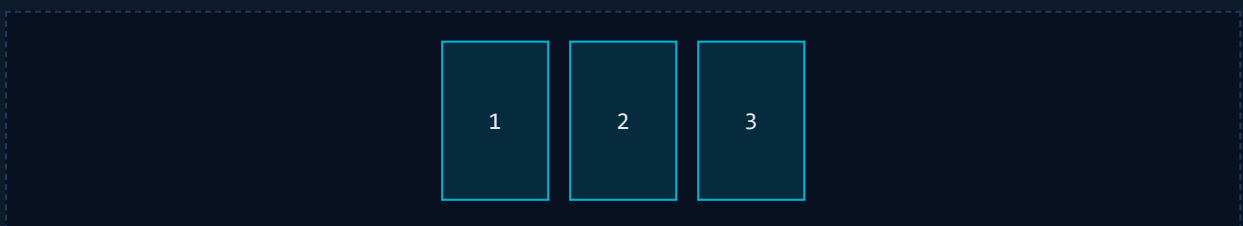
justify-content main axis ke along free space distribute karta hai. Row direction mein ye mostly horizontal alignment lagta hai, column direction mein vertical alignment.

Value	Behavior
flex-start	Items main start par
center	Items center mein grouped
flex-end	Items main end par
space-between	First start, last end, beech equal gaps
space-around	Each item ke around space
space-evenly	All spaces exactly equal

JUSTIFY-CONTENT: FLEX-START



JUSTIFY-CONTENT: CENTER



JUSTIFY-CONTENT: SPACE-BETWEEN

1

2

3

align-items

Cross axis par items ko align karna

DEFINITION

align-items flex items ko cross axis par align karta hai. Row layout mein ye vertical alignment control karta hai. Column layout mein ye horizontal alignment control karta hai.

align items values

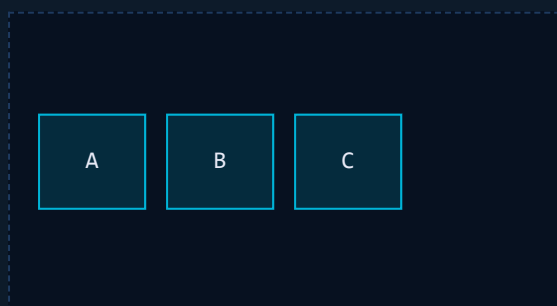
```
.container {  
  display: flex;  
  align-items: center;  
}
```

Value

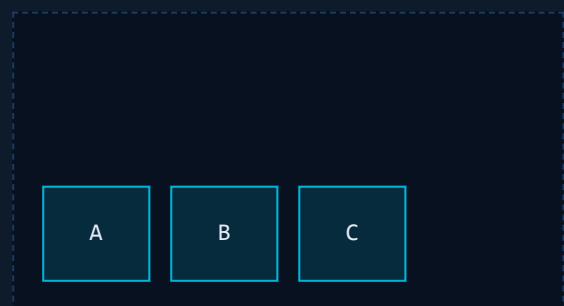
Meaning

stretch	Default, items cross axis fill karte hain
flex-start	Cross start par align
center	Cross axis center
flex-end	Cross end par align
baseline	Text baseline ke according align

ALIGN-ITEMS: CENTER



ALIGN-ITEMS: FLEX-END



■ **Centering Trick:** Perfect center ke liye parent par `display: flex; justify-content: center; align-items: center;` use hota hai.

gap and flex-wrap

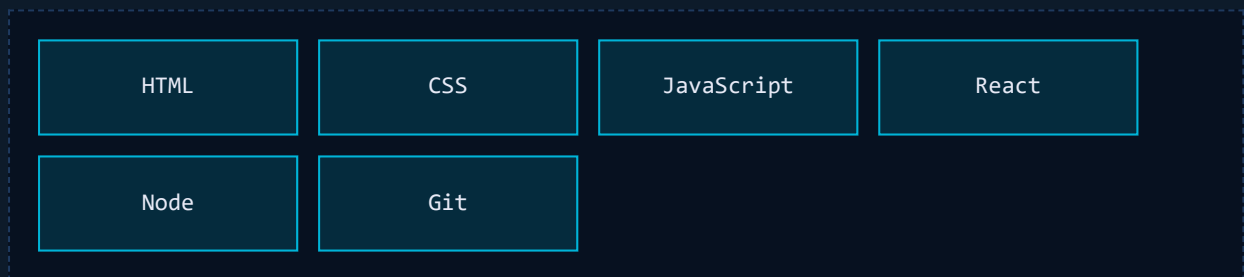
Items ke beech spacing aur next line wrapping

`gap` flex items ke beech consistent spacing create karta hai. Pehle margins se spacing manage karni padti thi; now gap cleaner hai. `flex-wrap` decide karta hai ki items ek line mein force honge ya space kam hone par next line par ja sakenge.

gap and wrap

```
.tags {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 10px;  
}
```

FLEX-WRAP: WRAP GRAPH



Property	Values	Use
<code>gap</code>	10px, 1rem, 12px 20px	Space between items
<code>row-gap</code>	16px	Rows ke beech space
<code>column-gap</code>	24px	Columns ke beech space
<code>flex-wrap</code>	nowrap, wrap, wrap-reverse	Items next line par jayenge ya nahi

align-content and align-self

Multiple flex lines aur individual item alignment

ADVANCED ALIGNMENT

`align-content` multiple flex lines ko cross axis par align karta hai. Ye tab useful hota hai jab `flex-wrap: wrap` ho aur container mein extra cross-axis space ho. `align-self` individual item ka cross-axis alignment override karta hai.

● ● ● align-content and align-self

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  align-content: space-between;  
}  
  
.item.special {  
  align-self: flex-end;  
}
```

Property	Applies To	Important Condition
<code>align-items</code>	All items in a line	Single or multiple lines
<code>align-self</code>	One item	Overrides align-items
<code>align-content</code>	Flex lines	Needs wrapping + extra cross space

■ **Common Mistake:** `align-content` single-line flex container par visible effect nahi deta. Multiple lines honi chahiye.

flex-grow, flex-shrink, flex-basis

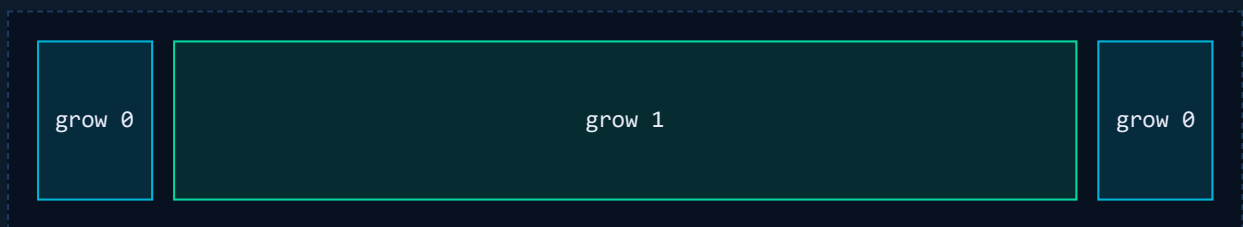
Flex items available space kaise consume ya shrink karte hain

SIZING FORMULA

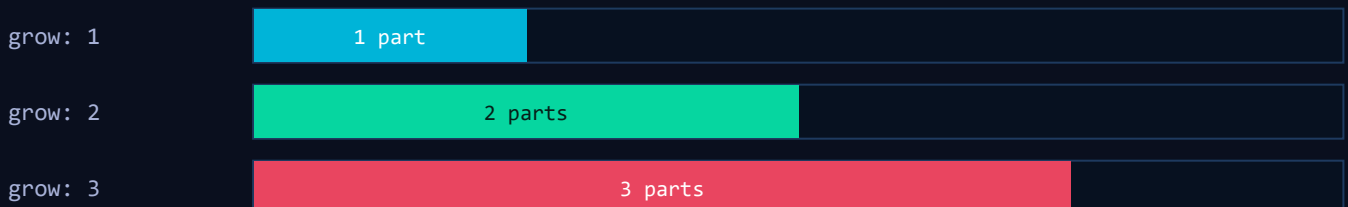
Flex item sizing teen important properties se control hota hai: `flex-grow`, `flex-shrink`, aur `flex-basis`. Ye decide karte hain item ka starting size, extra space distribution aur shortage mein shrinking behavior.

Property	Meaning	Default
<code>flex-grow</code>	Extra space mile to item kitna grow karega	0
<code>flex-shrink</code>	Space kam ho to item kitna shrink karega	1
<code>flex-basis</code>	Item ka initial main-axis size	auto

FLEX-GROW GRAPH



■ Grow Ratio Graph



grow shrink basis

```
.item {  
  flex-grow: 1;  
  flex-shrink: 1;  
  flex-basis: 200px;  
}
```

flex Shorthand

grow, shrink aur basis ko ek line mein set karna

DEFINITION

`flex` shorthand normally `flex-grow flex-shrink flex-basis` ko set karta hai. Professional CSS mein shorthand common hai because it clearly describes item sizing behavior.

flex shorthand

```
.item {  
  flex: 1 1 200px;  
  /* grow shrink basis */  
}
```

Shorthand	Meaning	Use
<code>flex: 1</code>	Grow equally, basis 0-like behavior	Equal columns
<code>flex: auto</code>	<code>flex: 1 1 auto</code>	Content-aware flexible item
<code>flex: none</code>	<code>flex: 0 0 auto</code>	Fixed natural size
<code>flex: 0 0 240px</code>	No grow, no shrink, 240px basis	Fixed sidebar
<code>flex: 1 1 300px</code>	Flexible, starts at 300px	Responsive cards

FLEX SHORTHAND SIZING GRAPH



■ **Best Habit:** Advanced layouts mein `flex: 1 1 280px` jaisi explicit shorthand values use karo, taaki grow/shrink/basis behavior clear rahe.

order and Auto Margins

Visual order aur available space ko smartly control karna

■ order Property

`order` flex items ka visual order change karta hai. Default value 0 hoti hai. Lower order pehle dikhta hai, higher order baad mein. Lekin HTML/source order same rehta hai.

● ● ● order example

```
.featured {  
  order: -1;  
}
```

■ Auto Margins in Flexbox

Flexbox mein auto margin available space absorb kar sakta hai. Navbar mein ek item ko right side push karne ke liye `margin-left: auto` very useful hai.

● ● ● navbar auto margin

```
.nav {  
  display: flex;  
  gap: 1rem;  
}  
  
.login {  
  margin-left: auto;  
}
```

■ **Accessibility Warning:** `order` sirf visual order change karta hai. Keyboard navigation and screen reader order source HTML ke according ho sakta hai. Important content order HTML mein correct rakho.

Real Project Flexbox Patterns

Navbar, card row, center box, sidebar and responsive components

■ Perfect Center

● ● ● center pattern

```
.center {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

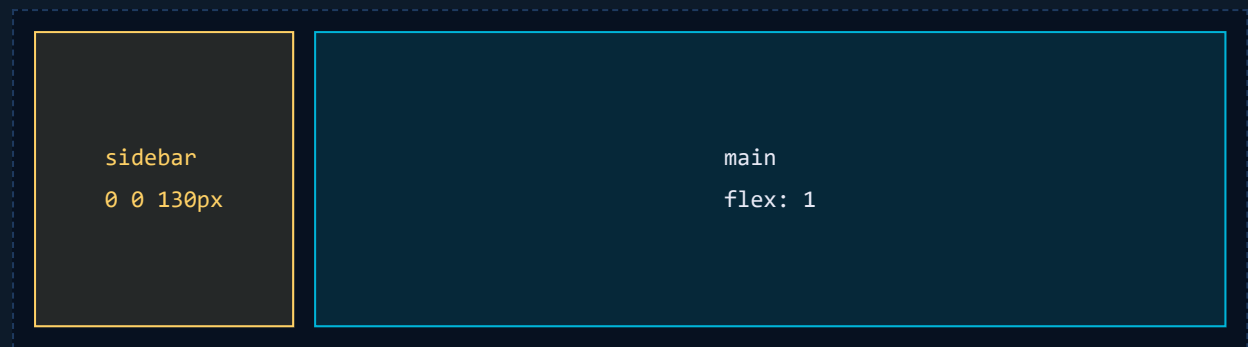
■ Navbar Pattern

● ● ● navbar flex

```
.navbar {  
  display: flex;  
  align-items: center;  
  gap: 1rem;  
}  
  
.nav-actions {  
  margin-left: auto;  
}
```

■ Sidebar Layout

SIDEBAR LAYOUT GRAPH



sidebar flex

```
.layout {  
  display: flex;  
  gap: 1.5rem;  
}  
  
.sidebar {  
  flex: 0 0 260px;  
}  
  
.main {  
  flex: 1 1 auto;  
}
```

Responsive Flexbox

Media queries, wrapping aur flexible basis se adaptive layouts

Flexbox responsive design ke liye powerful hai because items grow, shrink and wrap kar sakte hain.

Cards ke liye common pattern hai: `flex: 1 1 280px`. Iska matlab item grow karega, shrink karega, aur starting size 280px rakhega.

responsive cards

```
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
  
.card {  
  flex: 1 1 280px;  
}
```

■ Mobile to Desktop Direction Change

responsive direction

```
.feature {  
  display: flex;  
  flex-direction: column;  
  gap: 1rem;  
}  
  
@media (min-width: 768px) {  
  .feature {  
    flex-direction: row;  
    align-items: center;  
  }  
}
```

■ **Responsive Tip:** If layout is one-dimensional, Flexbox is usually enough. If you need rows and columns controlled together, CSS Grid is usually better.

Advanced Flexbox Behavior

min-width, overflow, nested flex aur equal height cards

■ min-width: 0 Problem

Flex items by default content ke intrinsic size ko respect kar sakte hain. Long text, code, URLs ya large content flex item ko shrink hone se rok sakta hai. Is case mein `min-width: 0` important fix hota hai.

● ● ● min-width fix

```
.content {  
  flex: 1 1 auto;  
  min-width: 0;  
}
```

■ Equal Height Cards

Flex items default `align-items: stretch` behavior ke through same row mein equal height feel de sakte hain. Card ke andar button bottom par push karne ke liye nested flex use hota hai.

● ● ● card button bottom

```
.card {  
  display: flex;  
  flex-direction: column;  
}  
  
.card-actions {  
  margin-top: auto;  
}
```

■ Flexbox vs Grid Decision

Need	Best Choice
Navbar, toolbar, button row	Flexbox
Centering one item	Flexbox or Grid
Card grid with rows and columns	Grid
Sidebar + flexible content	Flexbox or Grid
Complex page layout areas	Grid

Common Mistakes and Fixes

Flexbox bugs ko quickly identify karna

MISTAKE

justify-content ko always horizontal samajhna.
align-items ko always vertical samajhna.
Direct children concept bhoolna.
flex-wrap na lagana aur layout overflow karna.
flex: 1 ka basis behavior samjhe bina use karna.
Long text shrink issue mein min-width: 0 na lagana.
Full 2D grid layout Flexbox se force karna.

FIX

Axis first identify karo.
flex-direction ke according alignment samjho.
Parent direct children ko inspect karo.
Responsive rows ke liye flex-wrap use karo.
flex shorthand explicitly likho when needed.
Shrinking content ke liye min-width: 0 use karo.
Two-dimensional layout ke liye CSS Grid choose karo.

■ Debugging Checklist

Check	Question
Container	Kya parent par display: flex laga hai?
Direct child	Kya jis element ko flex item samajh rahe ho woh direct child hai?
Axis	Main axis row hai ya column?
Free space	justify-content ke liye actual free space available hai?
Wrapping	Items overflow kar rahe hain ya wrap karne chahiye?
Sizing	grow, shrink, basis expected values par hain?
Overflow text	Flex item ko min-width: 0 chahiye?

■ **Big Rule:** Flexbox ko blindly properties yaad karke use mat karo. Pehle axis, then container properties, then item sizing samjho.

CSS Flexbox - Complete Summary

Revision ke liye all important Flexbox properties ek jagah

■ Container Properties

Property	Meaning	Common Values
<code>display</code>	Flex container banana	flex, inline-flex
<code>flex-direction</code>	Main axis direction	row, column, row-reverse
<code>justify-content</code>	Main axis distribution	center, space-between, flex-end
<code>align-items</code>	Cross axis item alignment	stretch, center, flex-start
<code>flex-wrap</code>	Items next line par jayenge?	nowrap, wrap
<code>align-content</code>	Multiple flex lines alignment	space-between, center, stretch
<code>gap</code>	Items ke beech space	1rem, 12px 24px

■ Item Properties

Property	Meaning	Common Use
<code>flex-grow</code>	Extra space consume ratio	flexible columns
<code>flex-shrink</code>	Space shortage shrink ratio	responsive item shrink
<code>flex-basis</code>	Initial main-axis size	card min starting size
<code>flex</code>	grow shrink basis shorthand	flex: 1 1 280px
<code>align-self</code>	One item cross-axis override	special item alignment
<code>order</code>	Visual order control	carefully reorder items
<code>margin-left: auto</code>	Absorb free space	push nav item right

■ Fast Decision Guide

Need	Choose
Items one row mein align karne hain	display: flex
Horizontal distribution chahiye in row	justify-content
Vertical alignment chahiye in row	align-items
Items next line par wrap chahiye	flex-wrap: wrap
Equal flexible columns	flex: 1
Responsive card row	flex: 1 1 280px
One item right push karna	margin-left: auto
Complex rows and columns layout	CSS Grid

■ Best Practices - Yaad Rakhna

DO

Axis identify karke properties apply karo.
Container vs item properties separate yaad rakho.
Spacing ke liye gap use karo.
Responsive rows ke liye wrap use karo.
Advanced sizing ke liye flex shorthand samjho.
Long content issues mein min-width: 0 test karo.
One-dimensional UI ke liye Flexbox use karo.

AVOID

justify-content/align-items ko fixed horizontal/vertical samajhna.
Nested child ko flex item samajhna.
Margins se manual spacing hacks banana.
order se important content sequence change karna.
Flexbox se complex 2D layout force karna.
flex: 1 ka meaning samjhe bina use karna.
Overflow text problems ignore karna.

> NEXT TOPICS TO COVER

[CSS Grid](#)[Grid vs Flexbox](#)[Responsive Design](#)[Container Queries](#)

Pseudo Classes

Transitions

Animations

Forms Styling

Made with ❤ for Web Development Course learners
Shauray ke saath CSS seekhte raho 🚀 | Topic 15 - CSS Flexbox